Since we're mounting our app directory into the container, it's easy enough to delete the server.pid file manually:

```
$ rm tmp/pids/server.pid
```

This done, the Rails server should now start:

```
$ docker-compose up
```

You should see in the output that Rails is now up and running. However, this doesn't really solve the problem if it keeps happening. Luckily, we can put in place a workaround.

Let's see the workaround, and then discuss it:

1. Create a docker-entrypoint.sh file in your Rails root as follows:

```
#!/bin/sh
set -e

if [ -f tmp/pids/server.pid ]; then
  rm tmp/pids/server.pid
fi

exec "$@"
```

2. Make this file executable:

```
$ chmod +x docker-entrypoint.sh
```

3. Specify an ENTRYPOINT instruction in our Dockerfile file by adding the following line just before the final CMD instruction:

```
ENTRYPOINT ["./docker-entrypoint.sh"]
```

4. Stop, rebuild, and restart the web service:

```
$ docker-compose stop web
$ docker-compose build web
$ docker-compose up -d web
```

So what's all this doing?

An entrypoint is *prepended* to the command run upon starting a new container. In our case, we've set ./docker-entrypoint.sh as the ENTRYPOINT for the web service. This means, when we start a new web container, rather than simply running the default command of:

```
bin/rails s -b 0.0.0.0
```

it will actually run this, thanks to our new ENTRYPOINT instruction:

```
./docker-entrypoint.sh bin/rails s -b 0.0.0.0
```